



Manual de desarrolladores: portal, API, SDKs, tutoriales y webhooks

Dirigido a: **Desarrolladores e integradores técnicos (con rol Tenant Admin para acuñar tokens y configurar webhooks)**

Este manual explica cómo integrar sistemas externos con la plataforma agéntica multi-tenant a través del **Portal de desarrollador**, del **visor de documentación** del panel de administración y de las **pantallas de integración por proyecto** (webhooks entrantes y servidores MCP).

Recorrerás la página de inicio del portal, la referencia de la API pública v1 (OpenAPI 3.1 y Swagger UI), los SDKs oficiales de Python y TypeScript, los tutoriales paso a paso (acuñar un token, llamar a la API y configurar un webhook) y la guía de webhooks entrantes con firma HMAC. Después, dentro del área autenticada, verás la gestión visual de los webhooks entrantes de un proyecto (configuraciones, secreto mostrado una sola vez, rotación y registro de entregas) y la configuración de servidores MCP (catálogo de plantillas, credenciales vía Vault, prueba de conexión e importación de tools descubiertos).

El Portal de desarrollador es público (no requiere sesión) y no hace llamadas a la API: es una capa fina que enlaza el contrato OpenAPI, los READMEs de los SDKs y la documentación canónica bajo /docs. Las pantallas por proyecto, en cambio, requieren sesión con rol **Tenant Admin**.

Dos constantes de seguridad que verás repetidas en todo el manual: los **secretos se muestran una única vez** (token de API, secreto de firma de webhook) y después solo circulan metadatos; y el **aislamiento entre tenants** lo garantiza PostgreSQL RLS — un identificador de otro tenant devuelve un 404 limpio, indistinguible de un recurso inexistente.

Contenido

- 01** Visor de documentación (panel de administración)
- 02** Portal de desarrollador: inicio
- 03** API Reference: contrato OpenAPI y autenticación
- 04** SDKs oficiales: Python y TypeScript
- 05** Tutoriales: token, llamada a la API y webhook
- 06** Webhooks entrantes: orígenes, firma HMAC y checks
- 07** Webhooks entrantes: configuración por proyecto en el panel
- 08** Crear un webhook entrante: origen y reglas de mapeo (diálogo)
- 09** Servidores MCP del proyecto
- 10** Añadir un servidor MCP: plantillas, Vault y prueba de conexión (diálogo)

1 Visor de documentación (panel de administración)

Esta pantalla, dentro del área autenticada `/admin`, es el **visor de documentación** que permite explorar la documentación de cualquier proyecto del tenant en un solo lugar. La cabecera muestra el título **Documentación** con una breve descripción.

La columna izquierda tiene dos pestañas: **Explorar** y **Marcadores**. En **Explorar** encontrarás, de arriba a abajo: un buscador instantáneo de texto sobre los documentos, un panel de filtros por **categoría** y por **tipo** (que se activa al seleccionar un proyecto), y un árbol navegable de carpetas y archivos. Al seleccionar un documento, su ruta queda fijada en la URL (`?project=<id>&path=<ruta>`), de modo que el enlace es compatible y sobrevive a recargas.

Puedes marcar cualquier documento con la estrella desde el árbol, desde un resultado de búsqueda o desde la cabecera del visor; los marcados aparecen en la pestaña **Marcadores** (con un contador) y se guardan localmente en el navegador por tenant. El panel principal de la derecha renderiza el documento seleccionado en Markdown con su tabla de contenidos.

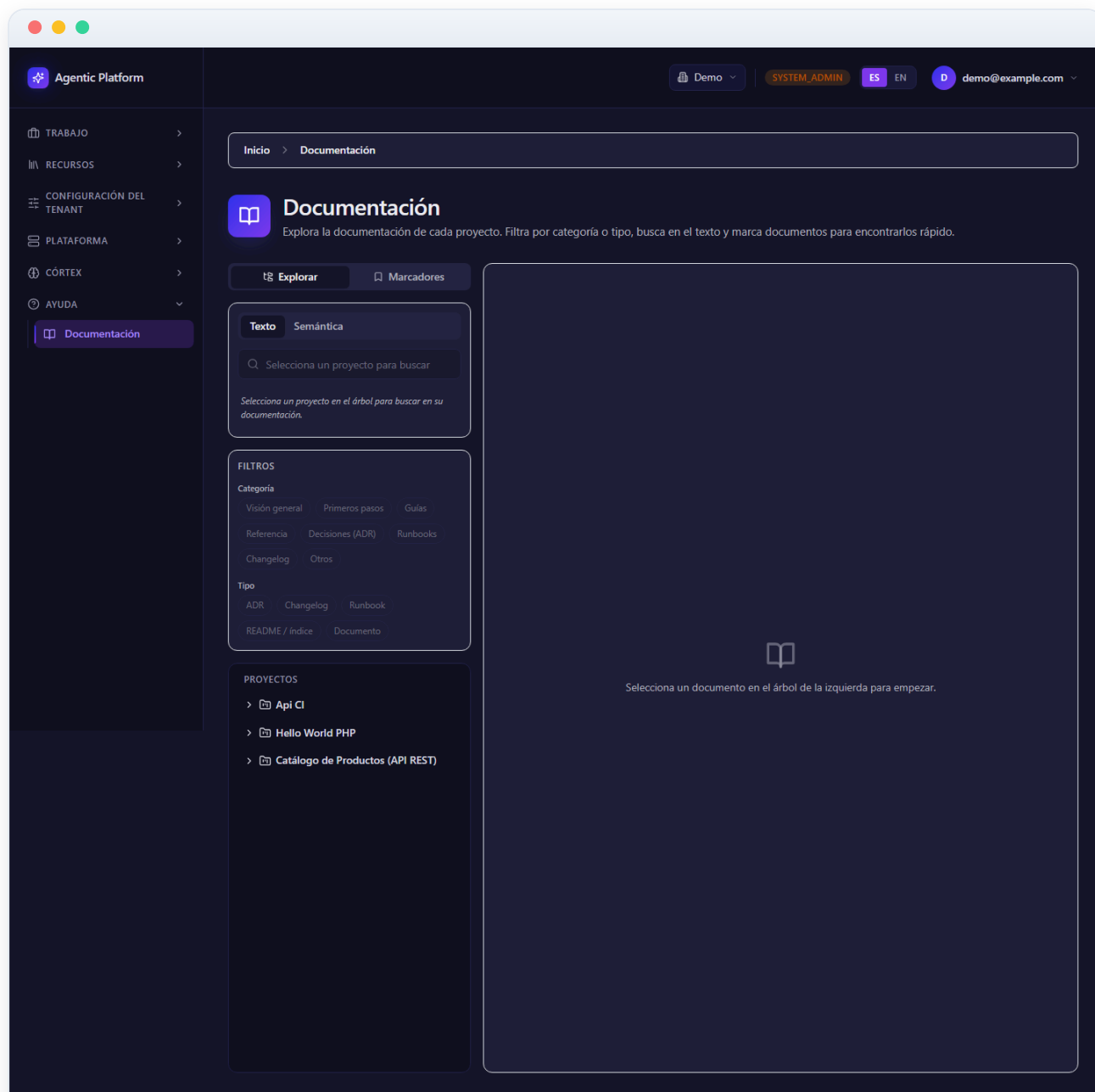


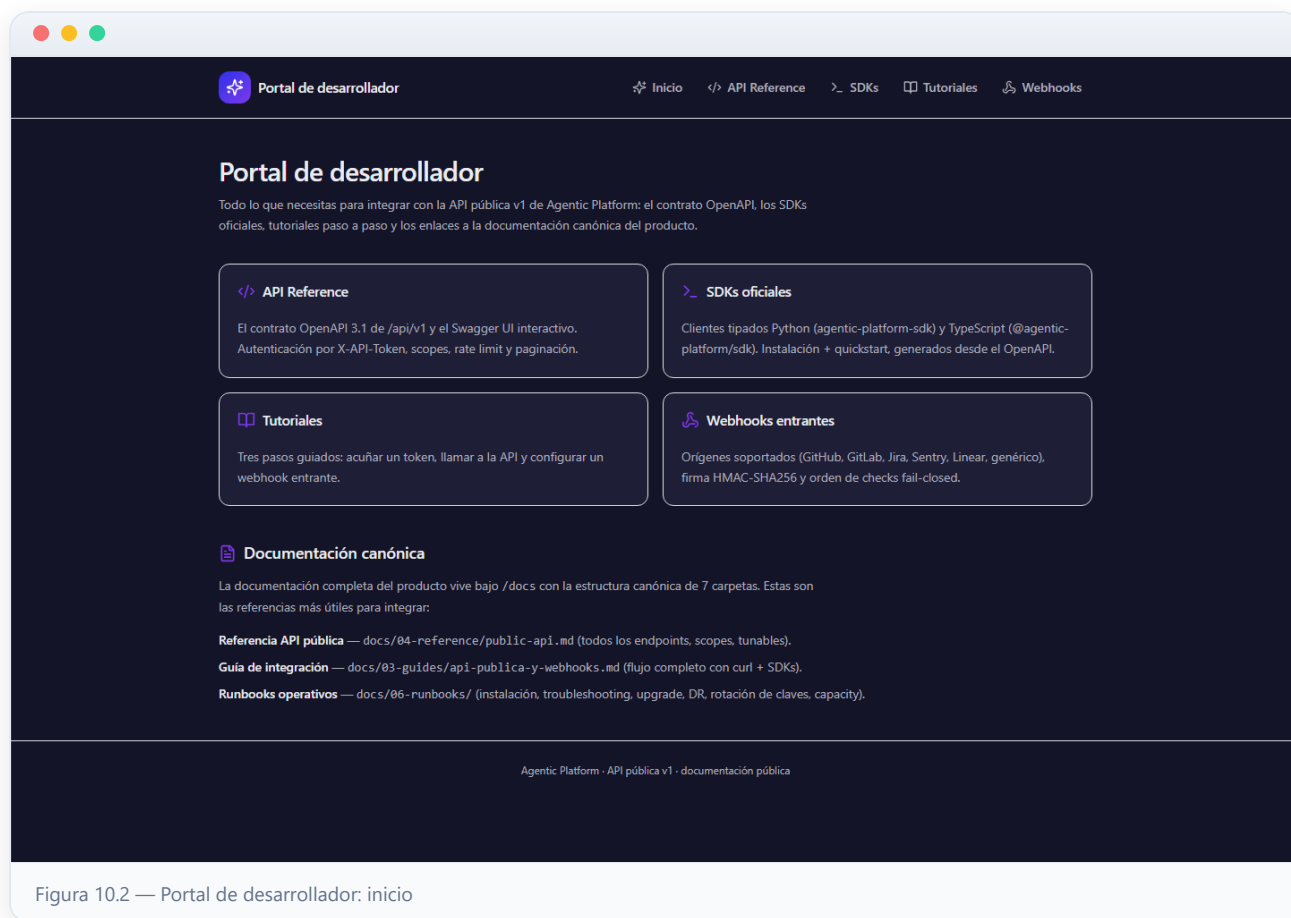
Figura 10.1 — Visor de documentación (panel de administración)

2 Portal de desarrollador: inicio

Esta es la página de entrada del **Portal de desarrollador**, una zona pública (fuera de `/admin`) que reúne todo lo necesario para integrar con la API pública v1. La cabecera incluye la marca **Portal de desarrollador** y una barra de navegación con: **Inicio**, **API Reference**, **SDKs**, **Tutoriales** y **Webhooks**.

El cuerpo presenta cuatro tarjetas enlazadas que resumen y dan acceso a cada sección: **API Reference** (contrato OpenAPI 3.1 y Swagger UI, autenticación por `X-API-Token`, scopes, rate limit y paginación), **SDKs oficiales** (clientes tipados de Python y TypeScript generados desde el OpenAPI), **Tutoriales** (tres pasos guiados) y **Webhooks entrantes** (orígenes soportados, firma HMAC-SHA256 y orden de checks).

Debajo, la sección **Documentación canónica** recuerda que la documentación completa del producto vive bajo `/docs` con la estructura de 7 carpetas y enlaza las referencias más útiles para integrar: la referencia API pública (`docs/04-reference/public-api.md`), la guía de integración (`docs/03-guides/api-publica-y-webhooks.md`) y los runbooks operativos (`docs/06-runbooks/`).



3 API Reference: contrato OpenAPI y autenticación

Esta página documenta la **API pública v1** (</api/v1>), una fachada fina y versionada sobre el dominio. La primera tarjeta, **Contrato OpenAPI 3.1 + Swagger UI**, enlaza los dos recursos públicos que conviene leer antes de acuñar el primer token: </api/v1/openapi.json> (el contrato crudo, consumido por los SDKs) y </api/v1/docs> (el Swagger UI interactivo para explorar y probar). Debes sustituir la ruta por la URL pública de tu instalación.

La tarjeta **Autenticación, scope y aislamiento** explica las reglas clave: el token viaja siempre en la cabecera `X-API-Token` (nunca como query param); los `GET` requieren scope `read` y los `POST` requieren `write` (un `write` no concede `read` implícito); el aislamiento entre tenants lo garantiza PostgreSQL RLS (un id ajeno devuelve un `404` limpio); hay un rate limit por token (100 req/min por defecto) con cabeceras `X-RateLimit-*`; y todas las listas son paginadas con `limit` / `offset`. Incluye un ejemplo `curl`.

Más abajo, la tabla **Endpoints (scope mínimo)** lista las rutas disponibles (projects, plans, tasks, conversations y knowledge bases) con su método y scope requerido, y la tabla **Códigos de estado** describe el significado de 200/201, 400, 401, 403, 404 y 429.

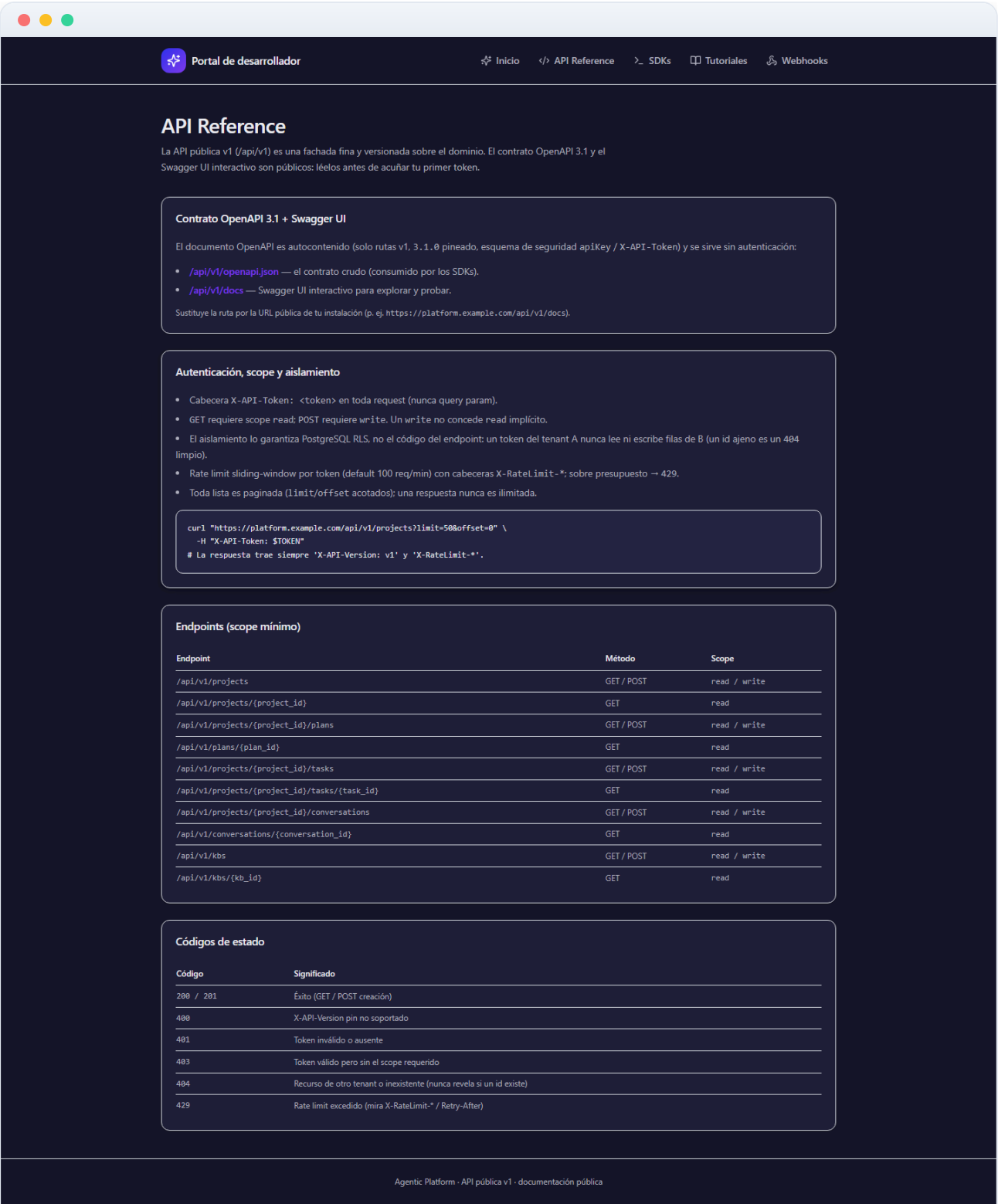


Figura 10.3 — API Reference: contrato OpenAPI y autenticación

4 SDKs oficiales: Python y TypeScript

Esta página describe los dos **SDKs oficiales** tipados sobre la API pública v1. Ambos se generan desde el contrato OpenAPI 3.1 en proceso (sin servidor vivo), de modo que siempre casan con el servidor: modelos generados más un cliente fino que fija el `X-API-Token` una vez y eleva errores tipados.

La tarjeta **SDK Python — `agentic-platform-sdk`** indica sus dependencias de runtime (`httpx` + `pydantic` v2), muestra el bloque de **Instalación** (`pip install` desde el monorepo o desde el registro interno) y un **Quickstart** con código de ejemplo para listar, obtener y crear proyectos y para consultar plans, tasks (project-scoped) y knowledge bases (tenant-scoped). Una respuesta non-2xx eleva `ApiError` con `status_code` y `body` para ramificar por 401, 403, 404 o 429.

La tarjeta **SDK TypeScript — `@agentic-platform/sdk`** destaca que tiene cero dependencias de runtime (usa `fetch` nativo, Node 18+ o navegador) y muestra su instalación y quickstart equivalentes. La última tarjeta, **Regeneración (reproducibilidad)**, explica cómo regenerar los modelos tras cambiar el contrato ejecutando los scripts `generate` de cada SDK.

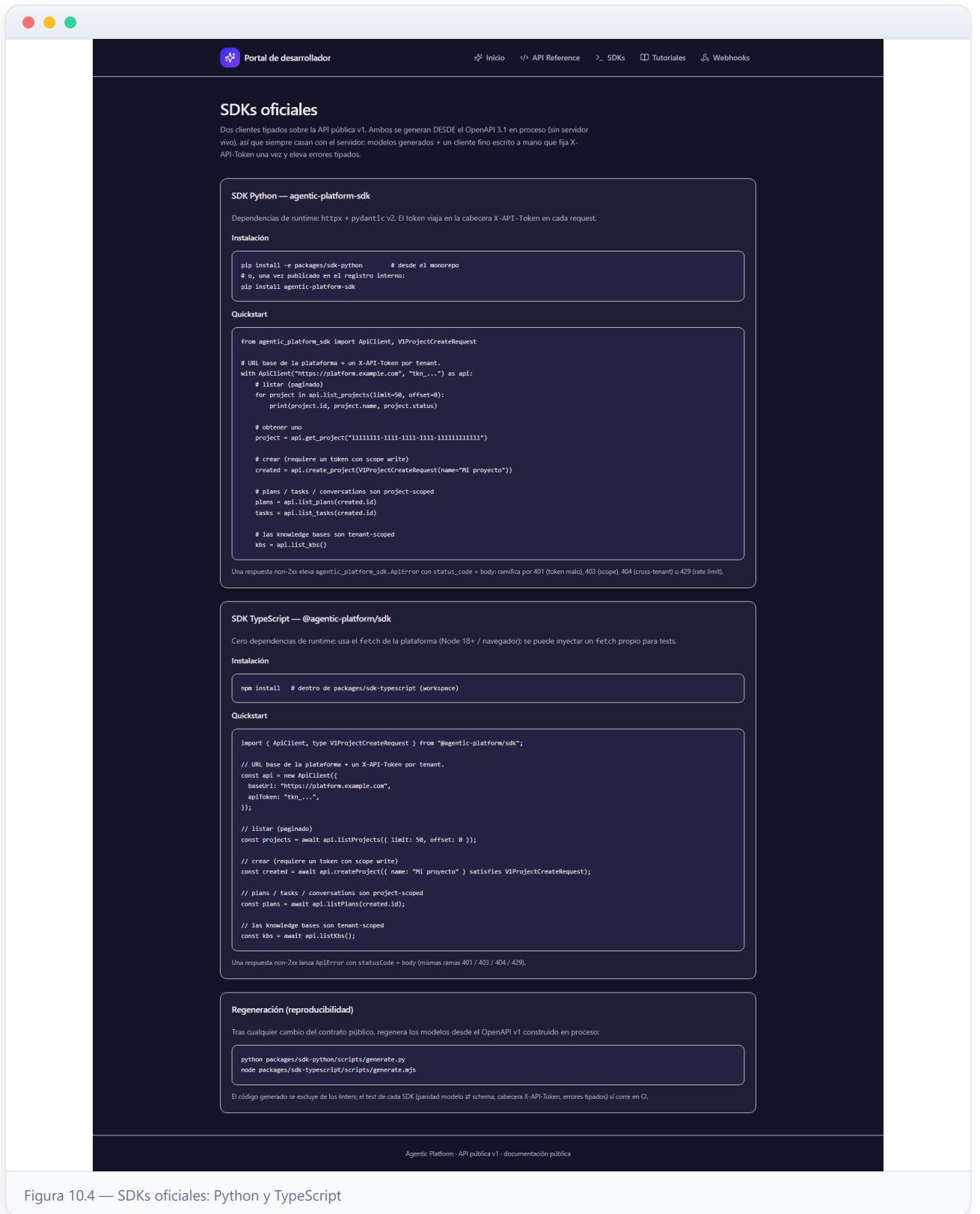


Figura 10.4 — SDKs oficiales: Python y TypeScript

5 Tutoriales: token, llamada a la API y webhook

Esta página recoge **tres tutoriales paso a paso** para pasar de cero a integrado. Requieren el stack en marcha y el rol **Tenant Admin** (acuñar tokens y gestionar webhooks lo exigen). Recuerda sustituir `platform.example.com` por la URL pública de tu instalación.

El paso **1 · Acuñar un X-API-Token** muestra cómo crear la credencial por tenant con `POST /auth/api-tokens`, indicando que el token claro se devuelve una sola vez (hay que guardarlo) y explicando los campos: `scopes` (`read` solo permite GET, añade `write` para crear), y los opcionales `rate_limit`, `expires_at` e `ip_allowlist`; la revocación se hace con `DELETE /auth/api-tokens/{token_id}`, efectiva de inmediato.

El paso **2 · Llamar a la API v1** muestra ejemplos `curl` para listar y crear proyectos con el token en la cabecera `X-API-Token`, recordando las reglas de scope y los códigos 404/429. El paso **3 · Configurar un webhook entrante** muestra cómo crear la configuración con `POST /projects/{project_id}/incoming-webhooks`, que devuelve el `incoming_path` y el `signing_secret` en claro una sola vez. La página enlaza a las secciones de SDKs y Webhooks para profundizar.



Figura 10.5 — Tutoriales: token, llamada a la API y webhook

6 Webhooks entrantes: orígenes, firma HMAC y checks

Esta página documenta los **webhooks entrantes**: un tool externo hace POST de un evento firmado con HMAC y el sistema lo verifica y lo mapea a una acción. El endpoint es público, por lo que **la propia firma HMAC actúa como autenticación**.

La tarjeta **Orígenes soportados (catálogo cerrado)** incluye una tabla con cada **origin** (github, gitlab, jira, sentry, linear y generic), la cabecera de firma que espera y los eventos típicos que generan acciones (crear tarea, comentar o escalar). Todas las firmas son HMAC-SHA256 sobre el body crudo, comparadas en tiempo constante; el endpoint público es **POST** `/webhooks/incoming/{origin}/{config_id}`.

La tarjeta **Orden de checks (fail-closed)** enumera los seis pasos que sigue el sistema: límite de tamaño del body (413), resolver la config (404), rate limit por config (429), verificar la firma HMAC (401, sin acción si falla), mapear y actuar, y persistir de forma idempotente. La última tarjeta, **Verificar la firma manualmente (genérico)**, muestra cómo calcular la firma con **openssl** y enviarla con **curl**, e indica que la rotación del secreto se hace con **POST** `.../incoming-webhooks/{config_id}/rotate-secret`, que invalida el secreto anterior de inmediato.

Webhooks entrantes

La dirección inversa del firmado saliente: un tool externo hace POST de un evento firmado con HMAC y el sistema lo verifica y lo mapea a una acción. El endpoint es público — la HMAC ES la autenticación.

Orígenes soportados (catálogo cerrado)

origin	Cabecera de firma	Eventos típicos
github	X-Hub-Signature-256: sha256=<hex>	push, PR review → crear/actualizar tarea
gitlab	X-Hub-Signature-256: sha256=<hex>	merge request → crear tarea
jira	X-Signature-256: <hex> (bare)	issue creado → crear tarea
sentry	X-Signature-256: <hex> (bare)	error → crear bug task / escalar
linear	X-Signature-256: <hex> (bare)	issue → crear tarea
generic	X-Signature-256: <hex> (bare)	integración a medida / proxy normalizador

Todas las firmas son HMAC-SHA256 sobre el body crudo, comparadas en tiempo constante. El endpoint público es POST `/webhooks/incoming/{origin}/{config_id}`.

Orden de checks (fail-closed)

- 1 - Body cap (413)** — Antes de leer el body (guarda anti-DDoS; default 1 MiB).
- 2 - Resolver config (404)** — `config_id` → fila. Inexistente / soft-deleted / deshabilitada / con origin que no casa → 404 (nunca revela si un id existe).
- 3 - Rate limit por config (429)** — Sliding-window keyed por `config_id` (default 120/min); una config nunca throttlea a otra.
- 4 - Verificar HMAC (401, SIN acción)** — Recomputa el MAC con el secreto por proyecto y compara en tiempo constante. Firma mala/ausente/manipulada → 401, nada persistido.
- 5 - Mapear + actuar** — Normaliza el payload (plantilla del origen) y resuelve `action_mappings` → acción (crear tarea / comentar / escalar), en la misma transacción que registra el evento.
- 6 - Persistir (idempotente)** — UNIQUE parcial (`config_id`, `delivery_id`); una redelivery colisiona, así que ni el evento ni su acción se reaplican.

Verificar la firma manualmente (genérico)

```
BODY='{"hello":"world"}'
SIG=$(printf '%s' "$BODY" | openssl dgst -sha256 -hmac "$SECRET" -hex | sed 's/^.* //' )
curl -X POST https://platform.example.com/webhooks/incoming/generic/$CONFIG_ID \
  -H "Content-Type: application/json" \
  -H "X-Signature-256: $SIG" \
  -d "$BODY"
# → 202 { "status": "accepted", "event_id": "...", "action": "create_task", "task_id": "..." }
```

Rotación: si sospechas que el secreto se filtró, POST `.../incoming-webhooks/{config_id}/rotate-secret` devuelve un nuevo claro una vez; el anterior deja de verificar de inmediato.

Agentic Platform · API pública v1 · documentación pública

Figura 10.6 — Webhooks entrantes: orígenes, firma HMAC y checks

7 Webhooks entrantes: configuración por proyecto en el panel

Además de la vía programática del tutorial (`POST /projects/{project_id}/incoming-webhooks`), el panel de administración ofrece una **pantalla de gestión visual** de los webhooks entrantes de cada proyecto, en Proyecto → Webhooks entrantes. Requiere rol **Tenant Admin** (la pantalla completa va tras un `RoleGuard` y el backend aplica RBAC + RLS de tenant y proyecto).

Cada configuración es una **tarjeta** con: el badge de su **origen** (GitHub, GitLab, Jira, Sentry, Linear o Genérico), el nombre, el estado *habilitado/deshabilitado*, la **URL completa del endpoint público** que debes registrar en la herramienta externa (el `incoming_path` relativo que devuelve el backend, prefijado con la URL base de la API de tu instalación) y la fecha del último evento recibido.

- **Editar**: cambia nombre, estado y reglas de mapeo (el origen es fijo tras crear).
- **Rotar secreto**: genera un secreto de firma nuevo y lo muestra una única vez; el anterior queda inválido de inmediato — recuerda actualizar el emisor externo en el mismo momento.
- **Eliminar**: baja lógica (soft-delete) de la configuración.
- **Entregas recientes**: despliega el registro de las últimas entregas recibidas — id de entrega del emisor, tipo de evento, si la firma se *verificó* y el timestamp de recepción. Es la primera parada para depurar por qué un webhook no dispara acciones.

Cuando creas o rotas, el **secreto de firma se muestra UNA sola vez** en un banner copiable en la parte superior; después, el listado solo devuelve metadatos. Es el mismo principio de secretos de toda la plataforma: si lo pierdes, no se puede recuperar — se rota.

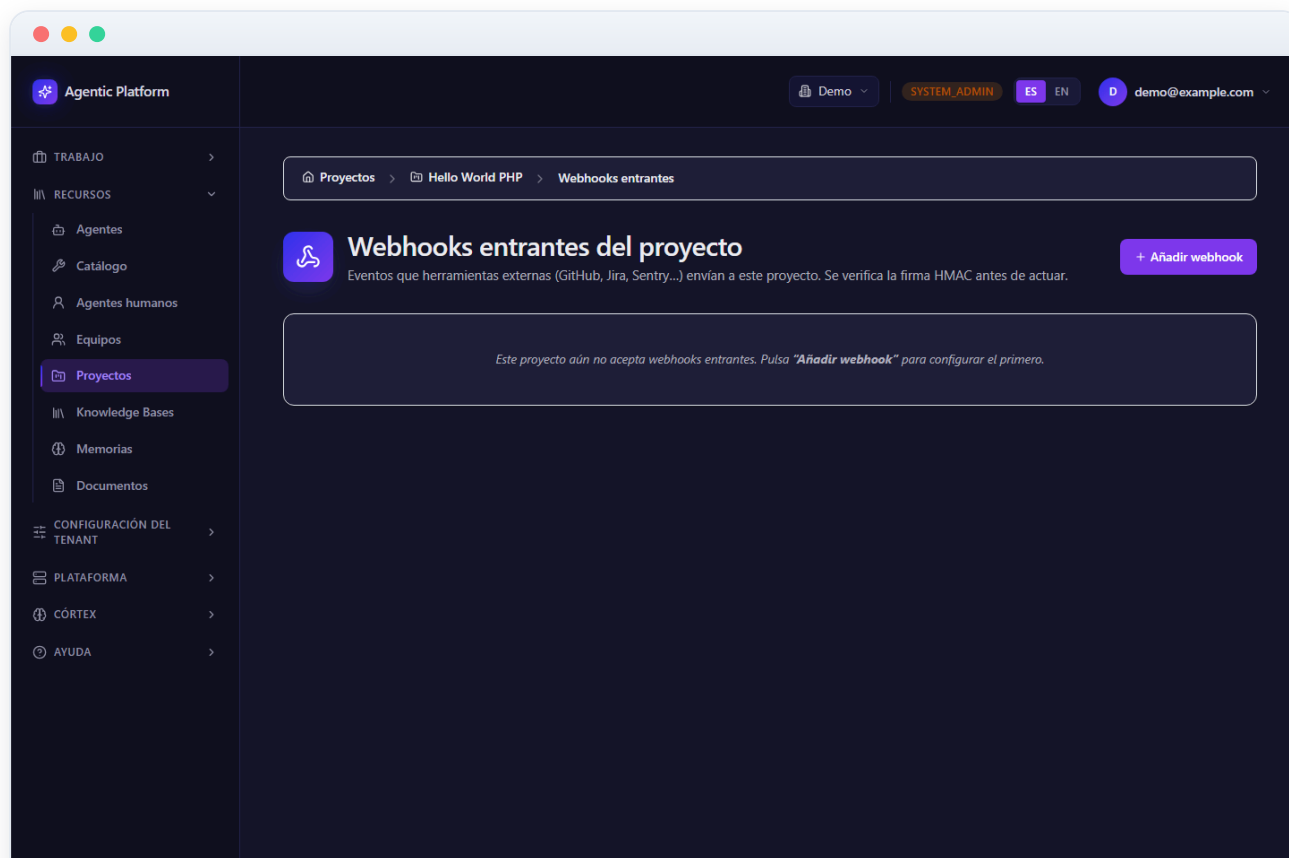


Figura 10.7 — Webhooks entrantes: configuración por proyecto en el panel

Crear un webhook entrante: origen y reglas de mapeo (diálogo)

El botón de añadir abre el diálogo de creación. Los campos base son el **origen** (el catálogo cerrado: GitHub, GitLab, Jira, Sentry, Linear o «Genérico (HMAC bare-hex)» para cualquier emisor propio), un **nombre** descriptivo y la casilla **habilitado**.

El corazón de la configuración son las **reglas de mapeo** (`action_mappings`): la lista ordenada que traduce eventos entrantes en acciones de la plataforma. Cada regla tiene:

- **Tipo de evento:** el `event_type` del payload a casar (o `*` como comodín para cualquier evento).
- **Acción:** qué hace la plataforma al recibirlo — **crear tarea**, **comentar tarea** o **escalar tarea**.
- **Plantillas opcionales** de título y cuerpo, para componer el texto de la tarea o del comentario a partir del evento.
- **Tarea destino** opcional, para las acciones de comentar/escalar sobre una tarea concreta.

Puedes añadir y quitar filas de reglas libremente antes de guardar. Al **crear**, la respuesta incluye el `signing_secret` en claro una única vez (banner copiable): configúralo inmediatamente en la herramienta emisora junto con la URL del endpoint. A partir de ahí, cada POST entrante se verifica contra ese secreto (HMAC-SHA256 sobre el body crudo, comparación en tiempo constante) siguiendo el orden de checks fail-closed descrito en la sección de Webhooks del portal.

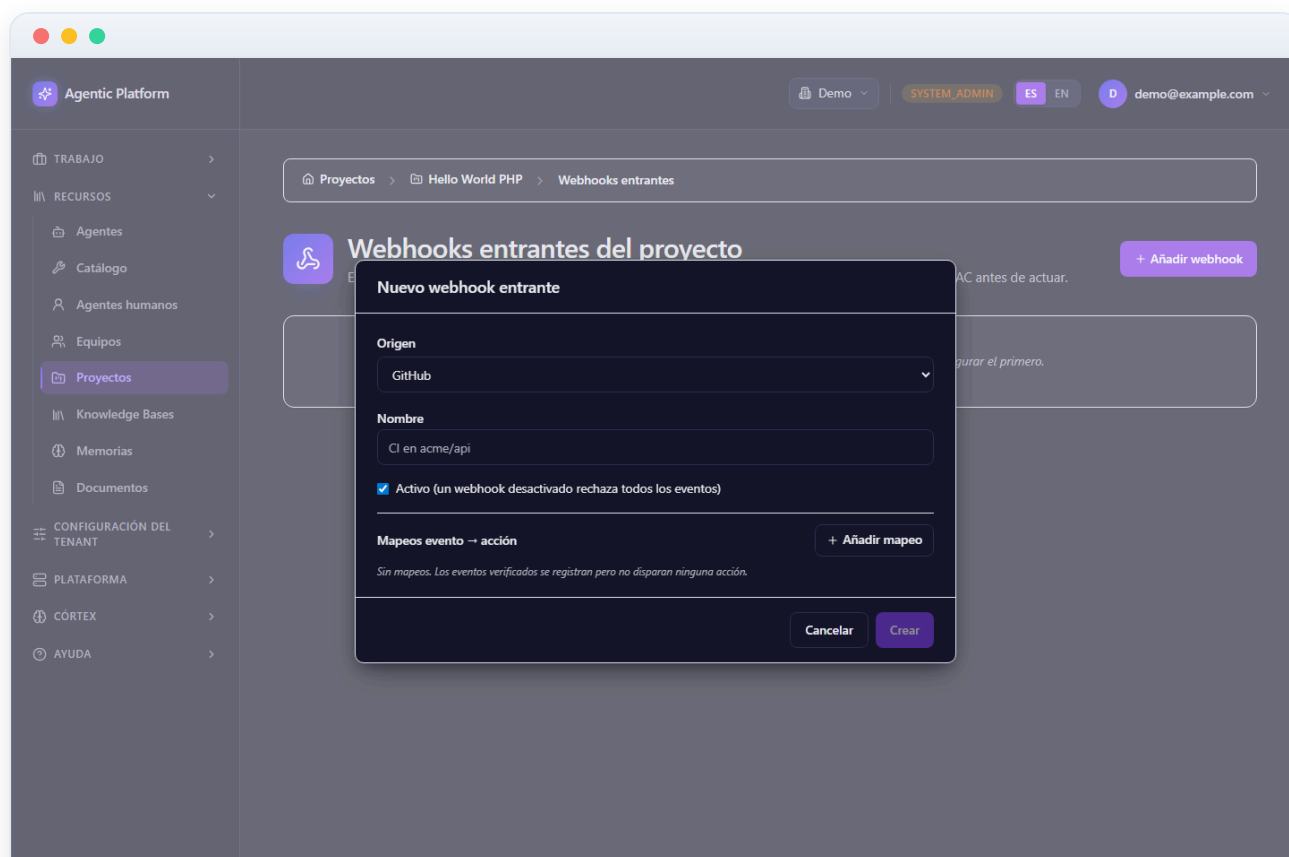


Figura 10.8 — Crear un webhook entrante: origen y reglas de mapeo (diálogo)

9 Servidores MCP del proyecto

Esta pantalla (Proyecto → MCP servers) gestiona los **servidores MCP** (Model Context Protocol) configurados en un proyecto: procesos o endpoints externos que exponen *tools* adicionales que los agentes del proyecto pueden usar (consultar un Jira, leer un Google Drive, lanzar un navegador...). La configuración vive en la propiedad `mcp_servers` del proyecto y se persiste reemplazando el array completo vía `PUT /projects/{id}`; el backend la valida con Pydantic y rechaza configuraciones inválidas (nombres duplicados, campos que no casan con el transporte, referencias de credencial sin el prefijo `vault:`).

Cada servidor es una **tarjeta** con su nombre, el badge del **transporte** — `stdio` (subproceso local), `sse` (stream HTTP) o `streamable_http` —, su indicador de autenticación y los botones de **editar** y **eliminar**. Según el transporte, la tarjeta muestra el comando y argumentos (stdio) o la URL (`sse/streamable_http`), además del timeout configurado.

Para integradores, esta es la vía recomendada de extender las capacidades de los agentes con sistemas propios: empaqueta tu integración como servidor MCP estándar y regístrala aquí; los agentes la descubren con namespacing `<server>.<tool>` (ADR 0052), sin tocar el código de la plataforma.

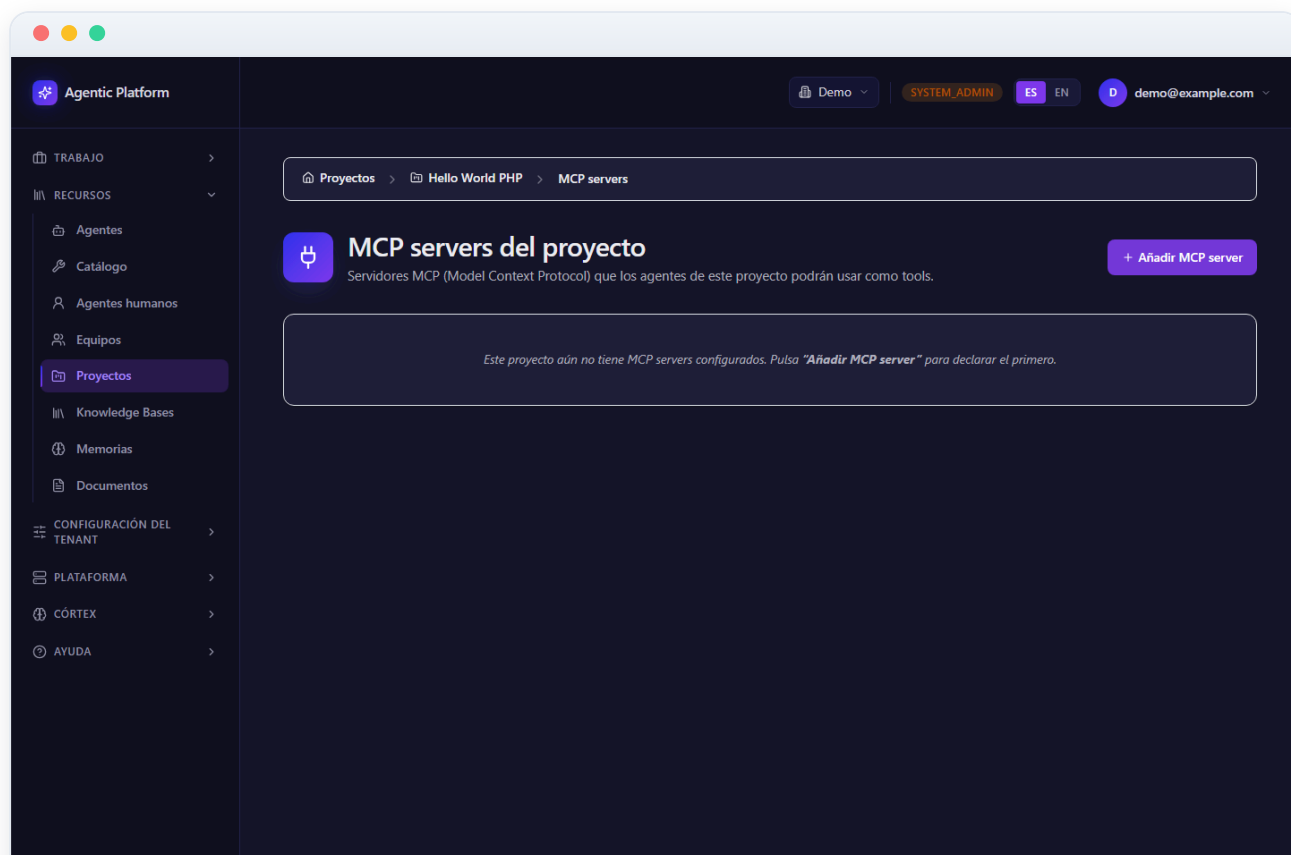


Figura 10.9 — Servidores MCP del proyecto

10 Añadir un servidor MCP: plantillas, Vault y prueba de conexión (diálogo)

El diálogo de alta arranca con un **selector de plantilla** alimentado por el catálogo verificado del backend (`GET /mcp-catalog` , 22 plantillas: GitHub, GitLab, Jira, Confluence, Google Drive, Gmail, Calendar, Slack, Teams, Discord, Notion, PostgreSQL, Sentry, Grafana, Brave, Tavily, Puppeteer, Memory, Sequential Thinking...), agrupadas por categoría (documentos, control de versiones, bases de datos, comunicación, issue trackers, observabilidad, búsqueda web, navegador...). Elegir una plantilla rellena automáticamente transporte, comando/argumentos o URL, variables de entorno estáticas y — si la plantilla declara secretos — el `auth_ref` ya **renderizado con el UUID del proyecto**: la ruta exacta de Vault donde tu administrador debe depositar la credencial, sin riesgo de teclearla mal.

Los campos también se pueden rellenar a mano: **nombre** (único en el proyecto), **transporte**, **comando + argumentos** (stdio) o **URL** (sse/streamable_http), tablas clave/valor de **env** y **headers**, el **timeout** en segundos y, en la sección avanzada, la credencial gestionada o el `auth_ref` crudo (siempre con prefijo `vault:` — la credencial en sí **nunca** se pega en esta configuración).

El botón «**Probar conexión**» conecta con el servidor y abre un panel inline con el resultado: nombre y versión del server, el número de **tools descubiertos** y su lista (cada tool con su descripción). Si el servidor falla, verás el error tipado para diagnosticarlo antes de guardar.

Desde ese mismo panel puedes **importar tools al catálogo del proyecto**: marca las que te interesen y pulsa «Importar N tools al catálogo». Se registran con origen *MCP*, con el nombre namespaced `<server>.<tool>` (para que `github.read_file` no se confunda con el `read_file` nativo) y en el nivel de riesgo «**Aislada**», de modo que quedan sujetas al flujo normal de asignación y aprobación de herramientas antes de que un agente pueda usarlas.

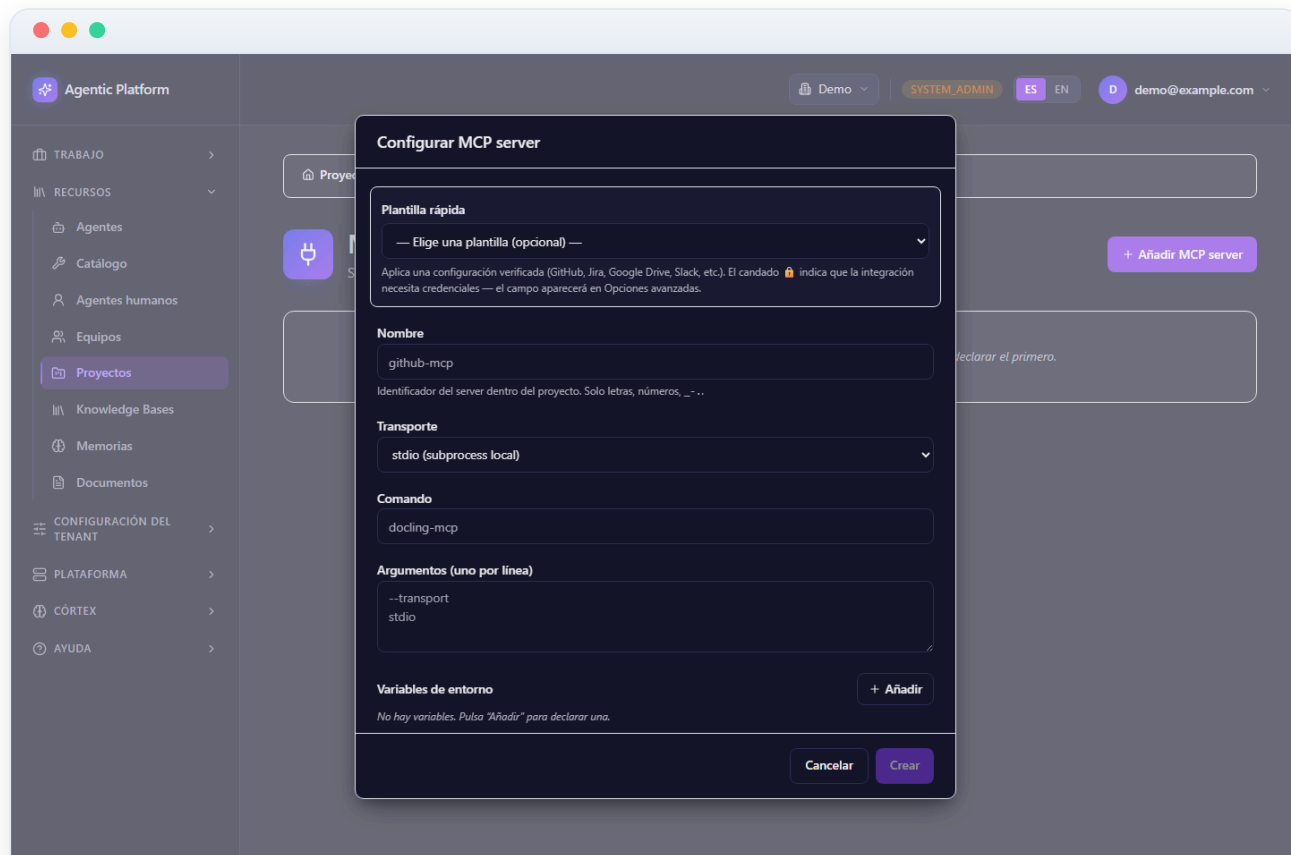


Figura 10.10 — Añadir un servidor MCP: plantillas, Vault y prueba de conexión (diálogo)